Goodwin, Graebe, Salgado.
Control System Design, Prentice Hall, 2001.

Chapter 23

MODEL PREDICTIVE CONTROL

23.1 Preview

As was mentioned in Chapter 11, all real-world control problems are subject to constraints of various types. The most common constraints are actuator constraints (amplitude and slew-rate limits). In addition, many problems also have constraints on state variables (e.g., maximal pressures that cannot be exceeded, minimum tank levels).

In many design problems, these constraints can be ignored, at least in the initial design phase. However, in other problems, these constraints are an inescapable part of the problem formulation, because the system operates near a constraint boundary. Indeed, in many process-control problems, the optimal steady-state operating point is frequently *at* a constraint boundary. In these cases, it is desirable to be able to carry out the design so as to include the constraints from the beginning.

Chapter 11 described methods (for dealing with constraints) based on anti-wind-up strategies. These are probably perfectly adequate for simple problems–especially SISO problems. Also, these methods can be extended to certain MIMO problems, as we shall see in Chapter 26. However, in more complex MIMO problems–especially those having both input and state constraints–it is frequently desirable to have a more formal mechanism for dealing with constraints in MIMO control-system design.

We describe here one such mechanism based on Model Predictive Control. This has actually been a major success story in the application of modern control. More than 2,000 applications of this method have been reported in the literature, predominantly in the petrochemical area. Also, the method is being increasingly used in electromechanical control problems, as control systems push constraint boundaries. Its main advantages are the following:

- It provides a *one-stop-shop* for MIMO control in the presence of constraints.
- It is one of the few methods that allow one to treat state constraints.

- Several commercial packages are available that give industrially robust versions of the algorithms aimed at chemical process control.

There are many alternative ways of describing MPC, including polynomial and state space methods. Here we will give a brief introduction to the method, using a state space description. This is intended to acquaint the reader with the basic ideas of MPC (e.g., the notion of receding-horizon optimization). It also serves as an illustration of some of the optimal control ideas introduced in Chapter 22, so as to provide motivation for MPC.

One of us (Graebe) has considerable practical experience with MPC in his industry, where it is credited with enabling very substantial annual financial savings through improved control performance. Another of us (Goodwin) has applied the method to several industrial control problems (including *added water* control in a sugar mill, rudder roll stabilization of ships, and control of a food extruder) where constraint handling was a key consideration.

Before delving into Model Predictive Control in detail, we pause to revisit the anti-wind-up strategies introduced in Chapter 11.

23.2 Anti-Wind-Up Revisited

We assume, for the moment, that the complete state of a system is directly measured. Then, if one has a time-invariant model for a system and if the objectives and constraints are time-invariant, it follows that the control policy should be expressible as a fixed mapping from the state to the control. That is, the optimal control policy will be expressible as

$$u_x^o(t) = h(x(t)) \tag{23.2.1}$$

for some static mapping $h(\circ)$. What remains is to give a characterization of the mapping $h(\circ)$. For general constrained problems, it will be difficult to give a simple parameterization to $h(\circ)$. However, we can think of the anti-wind-up strategies of Chapter 11 as giving a particular simple (ad-hoc) parameterization of $h(\circ)$. Specifically, if the control problem is formulated, in the absence of constraints, as a linear quadratic regulator, then we know from Chapter 22 that the unconstrained infinite horizon policy is of the form of Equation (23.2.1), where $h(\circ)$ takes the simple linear form

$$h^o(x(t)) = -K_\infty x(t) \tag{23.2.2}$$

for some constant-gain matrix K_∞. (See section §22.5.) If we then consider a SISO problem in which the control only is constrained, then the anti-wind-up form of (23.2.2) is (per section §18.9) given simply by

$$h(x(t)) = sat\{h^o(x(t))\} \tag{23.2.3}$$

Now, we found in Chapter 11 that the above scheme actually works remarkably well, at least for simpler problems.

We illustrate by a further simple example.

Example 23.1. *Consider a continuous-time double-integrator plant that is sampled at an interval $\Delta = 1$ second. The corresponding discrete-time model is of the form*

$$x(k+1) = \mathbf{A}x(k) + \mathbf{B}u(k) \tag{23.2.4}$$

$$y(k) = \mathbf{C}x(k) \tag{23.2.5}$$

where

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}; \quad \mathbf{B} = \begin{bmatrix} 0.5 \\ 1 \end{bmatrix}; \quad \mathbf{C} = \begin{bmatrix} 1 & 0 \end{bmatrix} \tag{23.2.6}$$

We choose to use infinite horizon LQR theory to develop the control law. Within this framework, we use the following weighting matrices:

$$\Psi = \mathbf{C}^T\mathbf{C}, \qquad \Phi = 0.1 \tag{23.2.7}$$

The unconstrained optimal control policy, as in (23.2.2)*, is then found via the methods described in Chapter 22.*

We next investigate the performance of this control law under different scenarios.

We first consider the case when the control is unconstrained.

Then, for an initial condition of $x(0) = (-10, 0)^T$, the output response and input signal are as shown in Figures 23.1 and 23.2 on the next page. Figure 23.3 on the following page shows the optimal response on a phase-plane plot.

We next assume that the input must satisfy the mild *constraint $|u(k)| \leq 5$. Applying the anti-wind-up policy* (23.2.3) *leads to the response shown in Figure 23.4 on page 743, with the corresponding input signal as in Figure 23.5 on page 743. The response is again shown on a phase-plane plot in Figure 23.6 on page 743.*

Inspection of Figures 23.4, 23.5, and 23.6 on page 743 indicates that the simple anti-wind-up strategy of Equation (23.2.3) *has produced a perfectly acceptable response in this case.*

□□□

The above example would seem to indicate that one need never worry about fancy methods. Indeed, it can be shown that the control law of Equation (23.2.3) is actually *optimal* under certain circumstances–see problem 23.10 at the end of the chapter. However, if we make the constraints more stringent, the situation changes, as we see in the next example.

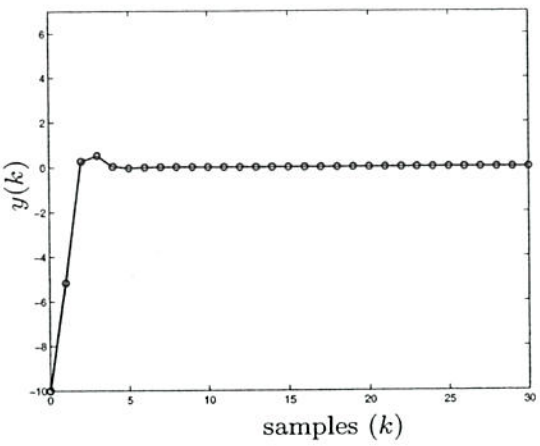

Figure 23.1. Output response without constraints

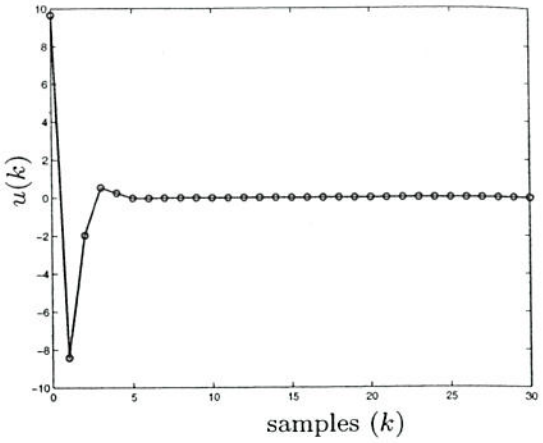

Figure 23.2. Input response without constraints

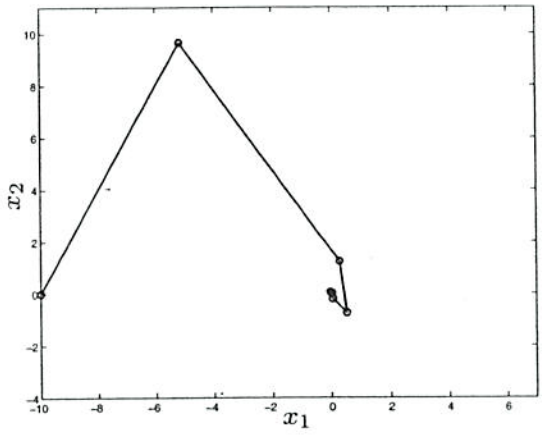

Figure 23.3. Phase-plane plot without constraints

Example 23.2. *Consider the same set-up as in Example 23.1, save that the input is now required to satisfy the more severe constraint* $|u(k)| \leq 1$. *Notice that this constraint is* 10% *of the initial unconstrained input shown in Figure 23.2. Thus, this is a relatively severe constraint. The control law of Equation* (23.2.3) *now leads to the response shown in Figure 23.7 on page 744, with corresponding input as in Figure 23.8 on page 744. The corresponding phase-plane plot is shown in Figure 23.9 on page 744.*

Inspection of Figure 23.7 on page 744 indicates that the simple policy of Equation (23.2.3) *is not performing well and, indeed, has resulted in large overshoot in this case.*

□□□

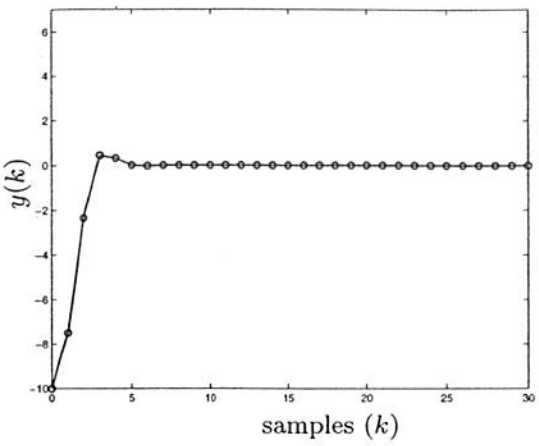

Figure 23.4. Output response with input constraint $|u(k)| \leq 5$

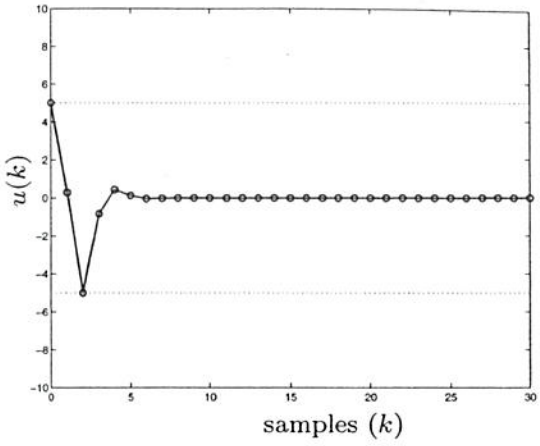

Figure 23.5. Input response with constraint $|u(k)| \leq 5$

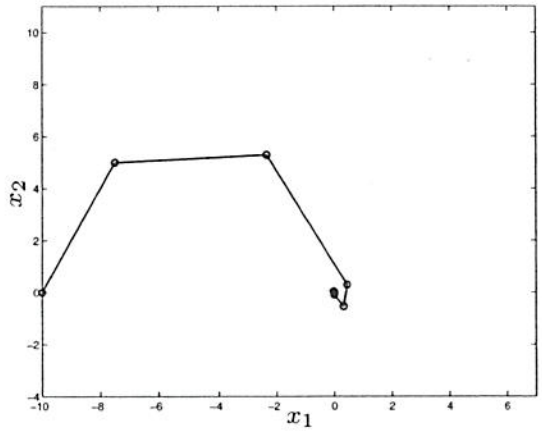

Figure 23.6. Phase-plane plot with constraint $|u(k)| \leq 5$

The reader may wonder what has gone wrong in the above example. A clue is given in Figure 23.9 on the following page. We see that the initial input steps have caused the velocity to build up to a large value. If the control were unconstrained, this large velocity would help us get to the origin quickly. However, because of the limited control authority, the system *braking capacity* is restricted, and hence large overshoot occurs. In conclusion, it seems that the control policy of Equation (23.2.3) has been too *shortsighted* and has not been able to account for the fact that *future* control inputs would be constrained as well as the current control input. The solution would seem to be to try to *look ahead* (i.e., predict the future response) and to take account of *current* and *future* constraints in deriving the control policy. This leads us to the idea of Model Predictive Control.

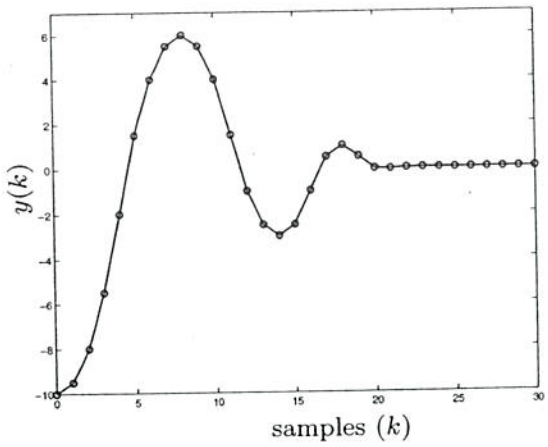

Figure 23.7. Output response with constraint $|u(k)| \leq 1$

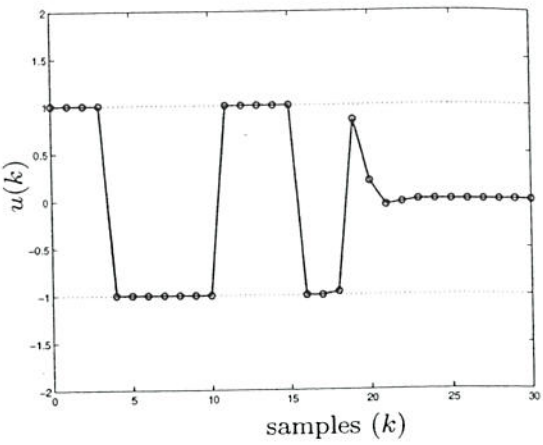

Figure 23.8. Input response with constraint $|u(k)| \leq 1$

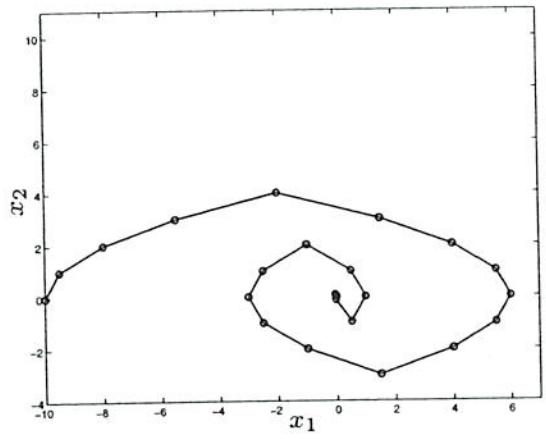

Figure 23.9. Phase-plane plot with constraint $|u(k)| \leq 1$

23.3 What is Model Predictive Control?

Model Predictive Control is a control algorithm based on solving an on-line *optimal* control problem. A *receding horizon* approach is used, which can be summarized in the following steps:

(i) At time k and for the current state $x(k)$, solve, on-line, an open-loop optimal control problem over some future interval, taking into account the *current* and *future* constraints.

(ii) Apply the first step in the optimal control sequence.

(iii) Repeat the procedure at time $(k+1)$, using the current state $x(k+1)$.

The solution is converted into a closed-loop strategy by using the measured value of $x(k)$ as the current state. When $x(k)$ is not directly measured, then one can obtain a closed-loop policy by replacing $x(k)$ by an estimate provided by some form of observer. The latter topic is taken up in section §23.6. For the moment, we assume that $x(k)$ is measured. Then, in a general nonlinear setting, the method is as follows:

Given a model

$$x(\ell+1) = f(x(\ell), u(\ell)), \qquad x(k) = x \tag{23.3.1}$$

the MPC at event (x, k) is computed by solving a constrained optimal control problem:

$$\mathcal{P}_N(x): \qquad V_N^o(x) = \min_{U \in \mathcal{U}_N} V_N(x, U) \tag{23.3.2}$$

where

$$U = \{u(k), u(k+1), \ldots, u(k+N-1)\} \tag{23.3.3}$$

$$V_N(x, U) = \sum_{\ell=k}^{k+N-1} L(x(\ell), u(\ell)) + F(x(k+N)) \tag{23.3.4}$$

and $\mathcal{U}_N$ is the set of U that satisfy the constraints over the entire interval $[k, k+N-1]$:

$$u(\ell) \in \mathbb{U} \qquad \ell = k, k+1, \ldots, k+N-1 \tag{23.3.5}$$

$$x(\ell) \in \mathbb{X} \qquad \ell = k, k+1, \ldots, k+N \tag{23.3.6}$$

together with the terminal constraint

$$x(k+N) \in W \tag{23.3.7}$$

Usually, $\mathbb{U} \subset \mathbb{R}^m$ is convex and compact, $\mathbb{X} \subset \mathbb{R}^n$ is convex and closed, and W is a set that can be selected appropriately to achieve stability.

In the above formulation, the model and cost function are time invariant. Hence, one obtains a time-invariant feedback control law. In particular, we can set $k = 0$ in the open-loop control problem without loss of generality. Then, at event (x, k), we solve

$$\mathcal{P}_N(x): \qquad V_N^o(x) = \min_{U \in \mathcal{U}_N} V_N(x, U) \tag{23.3.8}$$

where

$$U = \{u(0), u(1), \ldots, u(N-1)\} \tag{23.3.9}$$

$$V_N(x, U) = \sum_{\ell=0}^{N-1} L(x(\ell), u(\ell)) + F(x(N)) \tag{23.3.10}$$

subject to the appropriate constraints. Standard optimization methods are used to solve the above problem.

Let the minimizing control sequence be

$$U_x^o = \{u_x^o(0), u_x^o(1), \ldots, u_x^o(N-1)\} \tag{23.3.11}$$

Then the actual control applied at time k is the first element of this sequence, i.e.

$$u = u_x^o(0) \tag{23.3.12}$$

Time is then stepped forward one instant, and the above procedure is repeated for another N-step-ahead optimization horizon. The first input of the new N-step-ahead input sequence is then applied. The above procedure is repeated endlessly. The idea is illustrated in Figure 23.10 on the next page. Note that only the shaded inputs are actually applied to the plant.

The above MPC strategy *implicitly* defines a time-invariant control policy $h(\circ)$ as in (23.2.1)–i.e., a static mapping $h : \mathbb{X} \to \mathbb{U}$ of the form

$$h(x) = u_x^o(0) \tag{23.3.13}$$

The method originally arose in industry as a response to the need to deal with constraints. The associated literature can be divided into four generations, as follows:

- First Generation (1970's)–impulse or step-response linear models, quadratic cost function, and ad-hoc treatment of constraints.
- Second Generation (1980's)–linear state space models, quadratic cost function, input and output constraints expressed as linear inequalities, and quadratic programming used to solve the constrained optimal control problem.
- Third Generation (1990's)–several levels of constraints (soft, hard, ranked), mechanisms to recover from infeasible solutions.
- Fourth Generation (late 1990's)–nonlinear problems, guaranteed stability, and robust modifications.

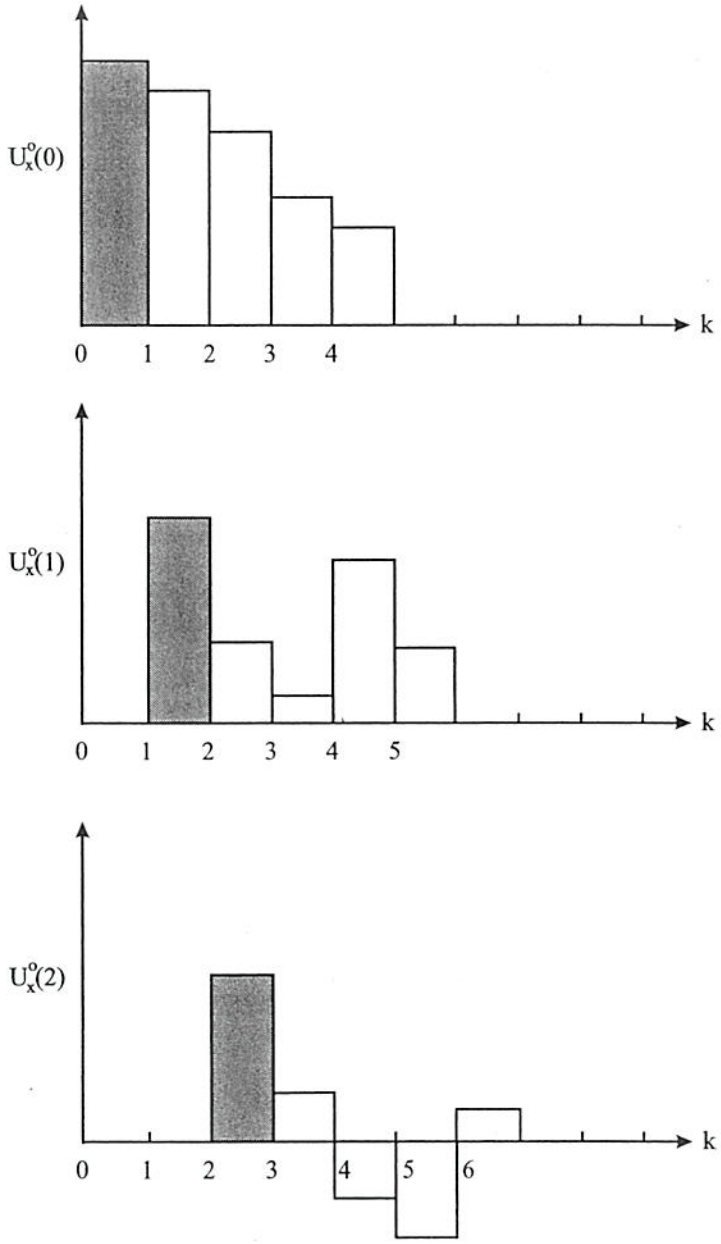

Figure 23.10. Receding-horizon control principle

23.4 Stability

A remarkable property of MPC is that one can establish stability of the resultant feedback system (at least with full state information). This is made possible by the fact that the value function of the optimal control problem acts as a Lyapunov function for the closed-loop system. We remind the reader of the brief introduction to Lyapunov stability given in Chapter 19.

For clarity of exposition, we make the following simplifying assumptions.

A1 : An additional constraint is placed on the final state of the receding-horizon optimization problem–namely, that

$$x(N) = 0 \tag{23.4.1}$$

where $x(N)$ is the terminal state resulting from the control sequence U_x^o as in (23.3.11).

A2 : $L(x, u)$ is positive definite in both arguments.

Theorem 23.1. *Consider the system* (23.3.1) *controlled by the receding-horizon MPC algorithm* (23.3.8) *to* (23.3.11) *and subject to the terminal constraint* (23.4.1). *This control law renders the resultant closed-loop system globally asymptotically stable.*

Proof

Under regularity assumptions, the value function $V_N^o(\cdot)$ is positive definite and proper: $V(x) \to \infty$ as $||x|| \to \infty$. It can therefore be used as a Lyapunov function for the problem. We recall that, at event (x, k), MPC solves

$$\mathcal{P}_N(x): \qquad V_N^o(x) = \min_{U \in \mathcal{U}_N} V_N(x, U) \tag{23.4.2}$$

$$V_N(x, U) = \sum_{\ell=0}^{N-1} L(x(\ell), u(\ell)) + F(x(N)) \tag{23.4.3}$$

subject to constraints.

We denote the optimal open-loop control sequence solving $\mathcal{P}_N(x)$ as

$$U_x^o = \{u_x^o(0), u_x^o(1), \ldots, u_x^o(N-1)\} \tag{23.4.4}$$

We recall that inherent in the MPC strategy is the fact that the actual control applied at time k is the first value of this sequence,

$$u = h(x) = u_x^o(0) \tag{23.4.5}$$

Let $x(1) = f(x, h(x))$, and let $x(N)$ be the terminal state resulting from the application of U_x^o. Note that we are assuming $x(N) = 0$.

A *feasible* solution (but not the optimal one) for the second step in the receding-horizon computation $\mathcal{P}_N(x_1)$, is then:

$$\tilde{U}_x = \{u_x^o(1), u_x^o(2), \ldots, u_x^o(N-1), 0\} \tag{23.4.6}$$

Then the increment of the Lyapunov function, upon using the true MPC optimal input and when moving from x to $x(1) = f(x, h(x))$, satisfies

$$\begin{aligned} \Delta_h V_N^o(x) &\triangleq V_N^o(x(1)) - V_N^o(x) \\ &= V_N(x(1), U_{x_1}^o) - V_N(x, U_x^o) \end{aligned} \tag{23.4.7}$$

However, $U_{x_1}^o$ is optimal, so we know that

$$V_N(x(1), U_{x_1}^o) \leq V_N(x(1), \tilde{U}_x) \tag{23.4.8}$$

where $\tilde{U}_x$ is the suboptimal sequence defined in (23.4.6). Hence, we have

$$\Delta_h V_N^o(x) \leq V_N(x(1), \tilde{U}_x) - V_N(x, U_x^o) \tag{23.4.9}$$

Using the fact the $\tilde{U}_x$ shares $(N-1)$ terms in common with U_x^o, we can see that the right-hand side of (23.4.9) satisfies

$$V_N(x(1), \tilde{U}_x) - V_N(x, U_x^o) = -L(x, h(x)) \tag{23.4.10}$$

where we have used the facts that U_x^o leads to $x(N) = 0$ (by assumption) and hence that $\tilde{U}_x$ leads to $x(N+1) = 0$.

Finally, using (23.4.9) and (23.4.10), we have

$$\Delta_h V_N^o(x) \leq -L(x, h(x)) \tag{23.4.11}$$

When $L(x, u)$ is positive definite in both arguments, then stability follows immediately from Theorem 19.1 on page 586.

□□□

Remark 23.1. *For clarity, we have used rather restrictive assumptions in the above proof, so as to keep it simple. Actually, both assumptions, (A1) and (A2), can be significantly relaxed. For example, assumption (A1) can be replaced by the assumption that $x(N)$ enters a terminal set in which "nice properties" hold. Similarly, assumption (A2) can be relaxed to requiring that the system be detectable in the cost function. We leave the interested reader to pursue these embellishments in the references given at the end of the chapter.*

□□□

Remark 23.2. *Beyond the issue of stability, a user of MPC would clearly also be interested in what, if any, performance advantages are associated with the use of this algorithm. In an effort to (partially) answer this question, we pause to revisit Example 23.2 on page 742.*

Example 23.3. *Consider again the problem described in Example 23.2, with input constraint $|u(k)| \leq 1$. We recall that the shortsighted policy used in Example 23.2 led to large overshoot.*

Here, we consider the cost function given in (23.3.10)*, with $N = 2$ and such that $F(x(N))$ is the optimal* unconstrained *infinite horizon cost and $L(x(\ell), u(\ell))$ is the incremental cost associated with the underlying LQR problem as described in Example 23.1.*

The essential difference between Example 23.2 and the current example is that, whereas in Example 23.2 we considered only the input constraint on the present control in the optimization, here we consider the constraint on the present and the next step. Thus, the derivation of the control policy is not quite as "short sighted" as was previously the case.

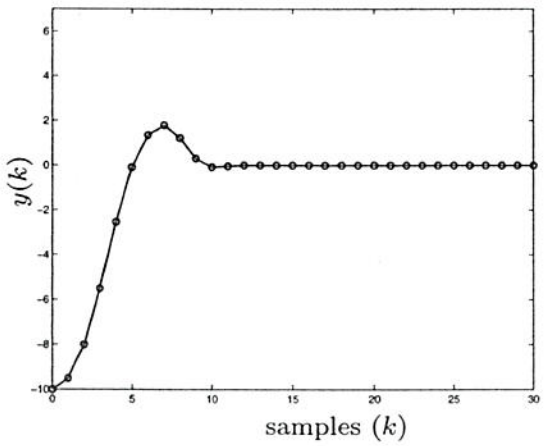

Figure 23.11. Output response when using MPC

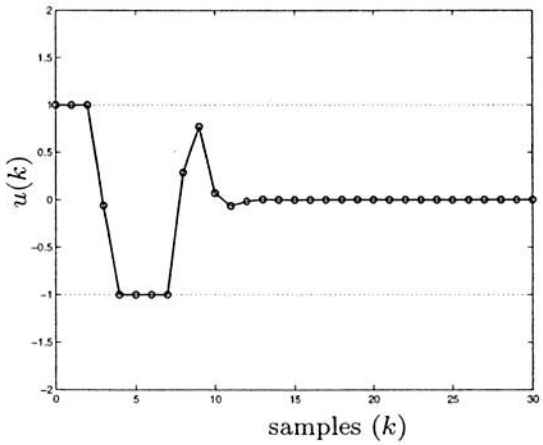

Figure 23.12. Input response when using MPC

Figure 23.11 shows the resultant response, and Figure 23.12 shows the corresponding input. The response is shown in phase-plane form in Figure 23.13. Com-

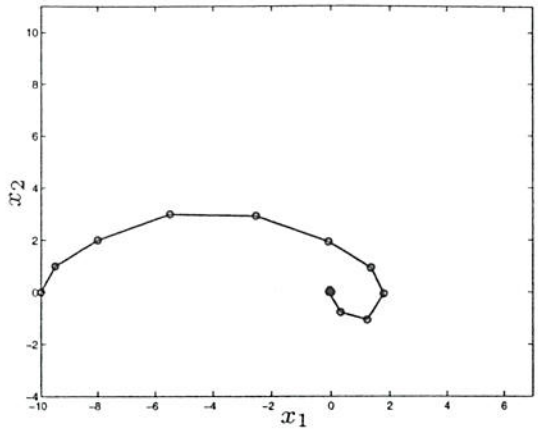

Figure 23.13. Phase-plane plot when using MPC

paring Figures 23.11 and 23.13 with Figures 23.7 and 23.9 on page 744 confirms the advantages of taking into account future constraints in deriving the control policy.

□□□

23.5 Linear Models with Quadratic Cost Function

So far, we have described the MPC algorithm in a rather general nonlinear setting. However, it might reasonably be expected that further insights can be obtained if one specializes the algorithm to cover the linear case with quadratic cost function. This will be the topic of the current section. Again, we will use a state space set-up. Let us therefore assume that the system is described by the following linear time-invariant model:

$$x(\ell+1) = \mathbf{A}x(\ell) + \mathbf{B}u(\ell) \tag{23.5.1}$$
$$y(\ell) = \mathbf{C}x(\ell) \tag{23.5.2}$$

where $x(\ell) \in \mathbb{R}^n$, $u(\ell) \in \mathbb{R}^m$, and $y(\ell) \in \mathbb{R}^m$.

We assume that $(\mathbf{A}, \mathbf{B}, \mathbf{C})$ is stabilizable and detectable and that 1 is not an eigenvalue of $\mathbf{A}$. So as to illustrate the principles involved, we go beyond the set-up described in section §23.2 to include reference tracking and disturbance rejection. Thus, consider the problem of tracking a constant set-point y_s and rejecting a time-varying output disturbance $\{d(\ell)\}$: we wish to regulate, to zero, the error

$$e(\ell) = y(\ell) + (d(\ell) - y_s) \tag{23.5.3}$$

It will be convenient in the sequel to make no special distinction between the output disturbance and the set-point, so we define an *equivalent* output disturbance d_e as the external signal:

$$d_e(\ell) = d(\ell) - y_s \tag{23.5.4}$$

Without loss of generality, we take the current time as 0.

Given knowledge of the external signal d_e and the current state measurement $x(0)$ (signals that will be later replaced by on-line estimates), our aim is to find the M-move control sequence $\{u(0), u(1), \ldots, u(M-1)\}$ that minimizes the finite-horizon performance index:

$$\begin{aligned} J_o =& [x(N) - x_s]^T \boldsymbol{\Psi}_f [x(N) - x_s] \\ &+ \sum_{\ell=0}^{N-1} e^T(\ell) \boldsymbol{\Psi} e(\ell) \\ &+ \sum_{\ell=0}^{M-1} [u(\ell) - u_s]^T \boldsymbol{\Phi} [u(\ell) - u_s] \end{aligned} \tag{23.5.5}$$

where $\boldsymbol{\Psi} \geq 0$, $\boldsymbol{\Phi} > 0$, $\boldsymbol{\Psi}_f \geq 0$. Note that this cost function is a slight generalization of the one given in Equation (22.7.3) of Chapter 22.

In (23.5.5), N is the prediction horizon, $M \leq N$ is the control horizon.

We assume that $d_e(\ell)$ contains time-varying components as well as a constant component. Let $\bar{d}_e$ denote the constant (steady-state) component.

In equation (23.5.5), we then let u_s and x_s denote the corresponding steady-state values of u and x:

$$u_s = -[\mathbf{C}(\mathbf{I} - \mathbf{A})^{-1}\mathbf{B}]^{-1} \bar{d}_e \tag{23.5.6}$$

$$x_s = (\mathbf{I} - \mathbf{A})^{-1}\mathbf{B} u_s \tag{23.5.7}$$

The minimization of (23.5.5) is performed on the assumption that the control reaches its steady-state value after M steps–that is, $u(\ell) = u_s$, $\forall \ell \geq M$.

In the presence of constraints on the input and output, this dynamic optimization problem simply amounts to finding the solution of a nondynamic constrained quadratic program. Indeed, from (23.5.1), and using $\mathbf{B}u_s = (I - \mathbf{A})x_s$, we can write

$$X - X_s = \boldsymbol{\Gamma} U + \boldsymbol{\Lambda}\, x(0) - \overline{X}_s \tag{23.5.8}$$

where

$$X = \begin{bmatrix} x(1) \\ x(2) \\ \vdots \\ x(N) \end{bmatrix}; \quad X_s = \begin{bmatrix} x_s \\ x_s \\ \vdots \\ x_s \end{bmatrix}; \quad U = \begin{bmatrix} u(0) \\ u(1) \\ \vdots \\ u(M-1) \end{bmatrix}; \quad \Lambda = \begin{bmatrix} \mathbf{A} \\ \mathbf{A}^2 \\ \vdots \\ \mathbf{A}^N \end{bmatrix}; \tag{23.5.9}$$

$$\Gamma = \begin{bmatrix} \mathbf{B} & 0 & \dots & 0 & 0 \\ \mathbf{AB} & \mathbf{B} & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ \mathbf{A}^{M-1}\mathbf{B} & \mathbf{A}^{M-2}\mathbf{B} & \dots & \mathbf{AB} & \mathbf{B} \\ \mathbf{A}^{M}\mathbf{B} & \mathbf{A}^{M-1}\mathbf{B} & \dots & \mathbf{A}^2\mathbf{B} & \mathbf{AB} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ \mathbf{A}^{N-1}\mathbf{B} & \mathbf{A}^{N-2}\mathbf{B} & \dots & \dots & \mathbf{A}^{N-M}\mathbf{B} \end{bmatrix}; \quad \overline{X}_s = \begin{bmatrix} x_s \\ x_s \\ \vdots \\ x_s \\ \mathbf{A}x_s \\ \vdots \\ \mathbf{A}^{N-M}x_s \end{bmatrix}$$

Then, using (23.5.8), (23.5.9), and

$$\overline{\mathbf{\Psi}} = \text{diag}[\mathbf{C}^T\mathbf{\Psi}\mathbf{C}, \dots, \mathbf{C}^T\mathbf{\Psi}\mathbf{C}, \mathbf{\Psi}_f] \tag{23.5.10}$$

$$\overline{\mathbf{\Phi}} = \text{diag}[\mathbf{\Phi}, \dots, \mathbf{\Phi}] \tag{23.5.11}$$

$$U_s = \begin{bmatrix} u_s^T & u_s^T & \dots & u_s^T \end{bmatrix}^T \tag{23.5.12}$$

$$\tilde{D} = [d_e(0)^T - \bar{d}_e^T, d_e(1)^T - \bar{d}_e^T, \dots, d_e(N-1)^T - \bar{d}_e, 0]^T \tag{23.5.13}$$

$$Z = \text{diag}[\mathbf{C}^T\mathbf{\Psi}, \mathbf{C}^T\mathbf{\Psi}, \dots, \mathbf{C}^T\mathbf{\Psi}] \tag{23.5.14}$$

we can express (23.5.5) as

$$\begin{aligned} J_o =& e^T(0)\mathbf{\Psi}e(0) + (X - X_s)^T\overline{\mathbf{\Psi}}(X - X_s) + (U - U_s)^T\overline{\mathbf{\Phi}}(U - U_s) \\ &+ 2(X - X_s)^T Z\tilde{D} + \tilde{D}^T \,\text{diag}[\mathbf{\Psi}, \dots, \mathbf{\Psi}]\tilde{D} \\ =& \overline{J}_o + U^T\mathbf{W}U + 2U^T\mathbf{V} \end{aligned} \tag{23.5.15}$$

In (23.5.15), $\overline{J}_o$ is independent of U, and

$$\mathbf{W} = \mathbf{\Gamma}^T\overline{\mathbf{\Psi}}\mathbf{\Gamma} + \overline{\mathbf{\Phi}}, \qquad \mathbf{V} = \mathbf{\Gamma}^T\overline{\mathbf{\Psi}}\,[\mathbf{\Lambda}x(0) - \overline{X}_s] - \overline{\mathbf{\Phi}}U_s + \mathbf{\Gamma}^T Z\tilde{D}\,. \tag{23.5.16}$$

From (23.5.15), it is clear that, if the design is *unconstrained*, J_o is minimized by taking

$$U = -\mathbf{W}^{-1}\mathbf{V} \tag{23.5.17}$$

Next, we introduce constraints into the problem formulation.

Magnitude and rate constraints on both the *input* and the *output* of the plant can be expressed as follows:

$$u_{\min} \leq u(k) \leq u_{\max}; \quad k = 0, 1, \ldots, M-1 \tag{23.5.18}$$

$$y_{\min} \leq y(k) \leq y_{\max}; \quad k = 1, 2, \ldots, N-1 \tag{23.5.19}$$

$$\Delta u_{\min} \leq u(k) - u(k-1) \leq \Delta u_{\max}; \quad k = 0, 1, \ldots, M-1 \tag{23.5.20}$$

These constraints can be expressed as *linear* constraints on U of the form

$$LU \leq K \tag{23.5.21}$$

Remark 23.3. *As an illustration for the single-input single-output case, L and K take the following form:*

$$L = \begin{bmatrix} I \\ -I \\ D \\ -D \\ W \\ -W \end{bmatrix}; \quad K = \begin{bmatrix} \overline{U}_{\max} \\ \overline{U}_{\min} \\ \overline{Y}_{\max} \\ \overline{Y}_{\min} \\ \overline{V}_{\max} \\ \overline{V}_{\min} \end{bmatrix} \tag{23.5.22}$$

where

I is the $M \times M$ identity matrix (M is the control horizon).

W is the following $M \times M$ matrix

$$W = \begin{bmatrix} 1 & & & \\ -1 & \ddots & & 0 \\ & \ddots & \ddots & \\ 0 & & -1 & 1 \end{bmatrix} \tag{23.5.23}$$

D is the following $(N-1) \times M$ matrix

$$D = \begin{bmatrix} CB & & & \\ CAB & \ddots & & \\ \vdots & \ddots & \ddots & \\ \vdots & & \ddots & \ddots \\ CA^{M-1}B & \cdots & CAB & CB \\ CA^{M}B & \cdots & \cdots & CAB \\ \vdots & & & \\ CA^{N-2}B & \cdots & CA^{N-M}B & CA^{N-M-1}B \end{bmatrix} \tag{23.5.24}$$

$$\overline{U}_{\max} = \begin{bmatrix} u_{\max} \\ \vdots \\ u_{\max} \end{bmatrix}; \qquad \overline{U}_{\min} = \begin{bmatrix} -u_{\min} \\ \vdots \\ -u_{\min} \end{bmatrix} \tag{23.5.25}$$

$$\overline{V}_{\max} = \begin{bmatrix} u(0) + \Delta u_{\max} \\ \Delta u_{\max} \\ \vdots \\ \Delta u_{\max} \end{bmatrix}; \qquad \overline{V}_{\min} = \begin{bmatrix} -u(0) - \Delta u_{\min} \\ -\Delta u_{\min} \\ \vdots \\ -\Delta u_{\min} \end{bmatrix} \tag{23.5.26}$$

$$\overline{Y}_{\max} = \begin{bmatrix} y_{\max} - CAx_o - d_1 \\ \vdots \\ \vdots \\ y_{\max} - CA^{N-1}x_o - d_{N-1} \\ -\sum_{i=0}^{N-M-2} CA^i B u_s \end{bmatrix}; \quad \overline{Y}_{\min} = \begin{bmatrix} -y_{\min} + CAx_o + d_1 \\ \vdots \\ \vdots \\ -y_{\min} + CA^{N-1}x_o + d_{N-1} \\ +\sum_{i=0}^{N-M-2} CA^i B u_s \end{bmatrix} \tag{23.5.27}$$

and $u_{\max}$, $u_{\min}$, $\Delta u_{\max}$, $\Delta u_{\min}$, $y_{\max}$, $y_{\min}$ *are the upper and lower limits on* $u(k)$, $(u(k) - u(k-1))$, *and* $y(k)$ *respectively.*

□□□

The optimal solution

$$U^{OPT} = \left[(\bar{u}^{OPT}(0))^T, (\bar{u}^{OPT}(1))^T, \ldots, (\bar{u}^{OPT}(M-1))^T\right]^T \tag{23.5.28}$$

can be numerically computed via the following static-optimization problem:

$$U^{OPT} = \arg \min_{\substack{U \\ LU \leq K}} U^T \mathbf{W} U + 2U^T \mathbf{V} \tag{23.5.29}$$

That this optimization is a convex problem is due to the quadratic cost and linear constraints. Also, standard numerical procedures (called Quadratic Programming algorithms or QP for short) are available to solve this subproblem. Many software packages (e.g., MATLAB) provide standard tools to solve QP problems. Thus, we will not dwell on this aspect here.

Note that we apply only the first control, $u^{OPT}(0)$. Then the whole procedure is repeated at the next time instant, with the optimization horizon kept constant.

Remark 23.4. *In the above development, we have assumed that 1 is not an eigenvalue of A. There are several ways to treat the case when 1 is an eigenvalue of A. For example, in the single-input single-output case, when A has a single eigenvalue at 1, then $u_s = 0$. Also, we can write the state space model so that the integrator is shifted to the output, i.e.,*

$$\begin{aligned} \bar{x}(k+1) &= \overline{A}\bar{x}(k) + \overline{B}u(k) \\ x'(k+1) &= \overline{C}\bar{x}(k) + x'(k) \\ y(k) &= x'(k) \end{aligned} \tag{23.5.30}$$

where $\overline{A}$, $\overline{B}$, and $\overline{C}$ correspond to the state space model of the reduced-order plant, i.e., the plant without the integrator.

With this transformation, $[\overline{A} - I]$ is nonsingular, and hence

$$\begin{bmatrix} \bar{x} \\ x' \end{bmatrix}_s = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ -\bar{d}_e \end{bmatrix} \tag{23.5.31}$$

The MPC problem is then solved in terms of these transformed state variables. The reader is encouraged to consider more general cases.

□□□

23.6 State Estimation and Disturbance Prediction

In the above description of the MPC algorithm, we have assumed, for simplicity of initial exposition, that the state of the system (including all disturbances) was available. However, it is relatively easy to extend the idea to the case when the states are not directly measured. The essential idea involved in producing a version of MPC that requires only output data is to replace the unmeasured state by an on-line estimate provided by a suitable observer. For example, one could use a Kalman filter (see Chapter 22) to estimate the current state $x(k)$ from observations of the output and input $\{y(\ell), u(\ell); \ell = \ldots, k-2, k-1\}$. Disturbances can also

be included in this strategy by including the appropriate noise-shaping filters in a composite model, as described in subsection §22.10.4.

The MPC algorithm also requires that future states and disturbances be predicted, so that the optimization over the future horizon can be carried out. To do this we simply propagate the deterministic model obtained from the composite system and noise model as described in subsection §22.10.5–see, in particular, Equation (22.10.80).

23.6.1 One-Degree-of-Freedom Controller

A one-degree-of-freedom output-feedback controller is obtained by including the set-point in the quantities to be estimated. The resultant output-feedback MPC strategy is schematically depicted in Figure 23.14, where $G_o(z) = \mathbf{C}(z\mathbf{I} - \mathbf{A})^{-1}\mathbf{B}$ is the plant transfer function and the observer has the structure discussed in subsections §22.10.4 and §22.10.5. In Figure 23.14, QP denotes the quadratic program used to solve the constrained optimal control problem.

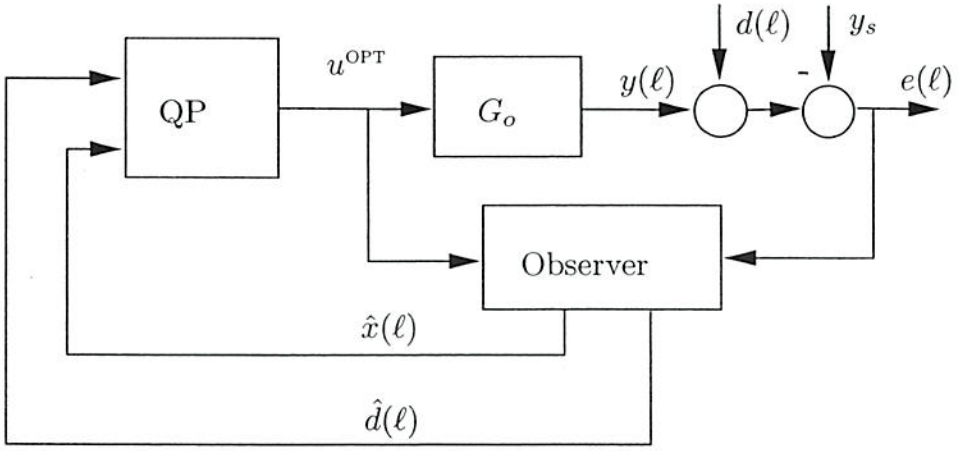

Figure 23.14. One-degree-of-freedom MPC architecture

23.6.2 Integral Action

An important observation is that the architecture described in Figure 23.14 gives a form of integral action. In particular, y is taken to the set-point y_s, irrespective of the true plant description (provided that a steady state is reached and that, in this steady state, u is not constrained). To show this, we argue as follows.

Say that $\{d(\ell)\}$ contains only constant values: then the observer needed to estimate these unknown constants will be of the form

$$\hat{d}_e(\ell + 1) = \hat{d}_e(\ell) + J_d(e(\ell) - \mathbf{C}\hat{x}(\ell) - \hat{d}_e(\ell)) \tag{23.6.1}$$

where $\{\hat{x}(\ell)\}$ denotes the estimate of the other plant states.

We see that, if a steady state is reached, then it must be true that $\hat{d}_e(\ell+1) = \hat{d}_e(\ell)$, and hence with $J_d \neq 0$ we have

$$e(\ell) = \mathbf{C}\hat{x}(\ell) + \hat{d}_e(\ell) \tag{23.6.2}$$

Now the control law has the property that the model output, $\mathbf{C}\hat{x}(\ell) + \hat{d}_e(\ell)$, is taken to zero in steady state. Thus, from (23.6.2), it must also be true that the actual quantity $\{e(\ell)\}$ as measured at the plant output must also be taken to zero in steady state. However, inspection of Figure 23.14 then indicates that, in this steady-state condition, the tracking error is zero.

23.7 Rudder Roll Stabilization of Ships

Here we present a realistic application of Model Predictive Control to rudder roll stabilization of ships. We will adapt the algorithm described in sections §23.5 and §23.6. The motivation for this particular case study follows.

It is desirable to reduce the rolling motion of ships (produced by wave action) so as to prevent cargo damage and improve crew efficiency and passenger comfort. Conventional methods for ship roll stabilization include water tanks, stabilization fins, and bilge keels. Another alternative is to use the rudder for roll stabilization as well as for course keeping. However, using the rudder simultaneously for course keeping and for roll reduction is nontrivial, because only one actuator is available to deal with two objectives. Therefore, frequency separation of the two models is required, and design trade-offs will limit the achievable performance. However, rudder roll stabilization is attractive because no extra equipment needs to be added to the ship.

An important issue is that the rudder mechanism is usually limited in amplitude and in slew rate. Hence, this is a suitable problem for Model Predictive Control.

In order to describe the motion of a ship, six independent coordinates are necessary. The first three coordinates and their time derivatives correspond to the position and translational motion; the other three coordinates and their time derivatives correspond to orientation and rotational motion. For marine vehicles, the six different motion components are named *surge, sway, heave, roll, pitch,* and *yaw.* The most generally used notation for these quantities are x, y, z, ϕ, θ, and ψ, respectively. Figure 23.15 shows the six coordinate definitions and the reference frame most generally adopted.

Figure 23.15 also shows the *rudder angle*, which is usually noted as δ.

The roll motion due to the rudder presents a faster dynamic response than does the rudder-produced yaw motion, because of the different moments of inertia of the hull associated with each direction. This property is exploited in rudder roll stabilization.

The common method of modeling the ship motion is based on the use of Newton's

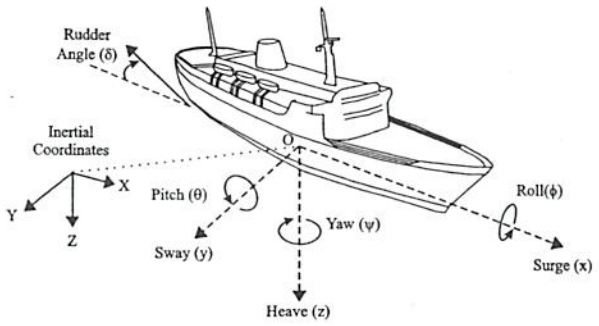

Figure 23.15. Magnitudes and conventions for ship motion description

equation in surge, sway, and roll. By using this approach, a nonlinear model is obtained, from which a linearized state space model can be derived that depends upon five states: $[v, r, p, \phi, \psi]^T$.

The model is a function of the ship speed. However, this is not of much concern; the model is quasi-constant over a reasonable speed range. This allows the use of gain scheduling to update the model according to different speeds of operation.

Undesired variations of the roll and yaw angles are generated mainly by waves. These wave disturbances are usually considered at the output, and the complete model of the ship dynamics, including the output equation, is of the form

$$\dot{x} = \mathbf{A}x + \mathbf{B}\delta \tag{23.7.1}$$

$$y = \mathbf{C}x + d_{wave} \tag{23.7.2}$$

Typically, only the roll and yaw are directly measured: $y := [\phi, \psi]^T$. In Equation (23.7.2), d_{wave} is the wave-induced disturbance on the output variables.

The wave disturbances can be characterized in terms of their frequency spectrum. This frequency spectrum can be simulated by using filtered white noise. The filter used to approximate the spectrum is usually a second-order one of the form

$$H(s) = \frac{K_w s}{s^2 + 2\xi\omega_o s + \omega_o^2} \tag{23.7.3}$$

where K_w is a constant describing the wave intensity, ξ is a damping coefficient, and ω_o is the dominant wave frequency. This model produces decaying *sine-wave-like* disturbances. The resultant colored noise representing the effect of wave disturbances is added to the two components of y, with appropriate scaling. (Actually, the wave motion affects mainly the roll motion.)

The design objectives for rudder roll stabilization are:

- Increase the damping and reduce the roll amplitude;

- Control the heading of the ship.

The rudder is constrained in amplitude and slew rate.

For merchant ships, the rudder rate is within the range from 3 to 4 deg/sec; for military vessels, it is in the range from 3 to 8 deg/sec. The maximum rudder excursion is typically in the range from 20 to 35 deg.

We will adopt the Model Predictive Control algorithm described in section §23.5.

The following model was used for the ship–see the references given at the end of the chapter.

$$\mathbf{A} = \begin{bmatrix} -0.1795 & -0.8404 & 0.2115 & 0.9665 & 0 \\ -0.0159 & -0.4492 & 0.0053 & 0.0151 & 0 \\ 0.0354 & -1.5594 & -0.1714 & -0.7883 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix} \tag{23.7.4}$$

$$\mathbf{B} = \begin{bmatrix} 0.2784 \\ -0.0334 \\ -0.0894 \\ 0 \\ 0 \end{bmatrix} \tag{23.7.5}$$

This system was sampled with a zero-order hold and sampling interval equal to 0.5 sec.

Details of the Model Predictive Control optimization criterion of Equation (23.4.6) were

$$\mathbf{\Psi} = \begin{bmatrix} 90 & 0 \\ 0 & 3 \end{bmatrix}; \qquad \mathbf{\Phi} = 0.1 \tag{23.7.6}$$

Optimization horizon, $N = 20$; Control horizon, $M = 18$.

Remark 23.5. *Note that the continuous time system* (23.7.4) *and* (23.7.5) *contains a pure integrator. Hence, the discretized system has an eigenvalue at 1; however, for this problem, we have taken the set-point on the output to be zero and the disturbance has zero mean. Thus, the problem reduces to a regulation problem where* $x_s = 0$, $u_s = 0$.

□□□

Only roll, ϕ, and yaw, ψ, were assumed to be directly measured, so a Kalman filter was employed to estimate the 5 system states and 2 noise states in the composite model. The output disturbance was predicted by using the methodology described in section §23.6.

The set-points for the two system outputs were taken to be zero, and there were no constant off-sets in the disturbance model. Thus, the problem reduces to a regulation problem.

In the following plots we show the roll angle (with and without rudder roll stabilization), the yaw angle (with and without rudder roll stabilization), and the rudder angle with roll stabilization.

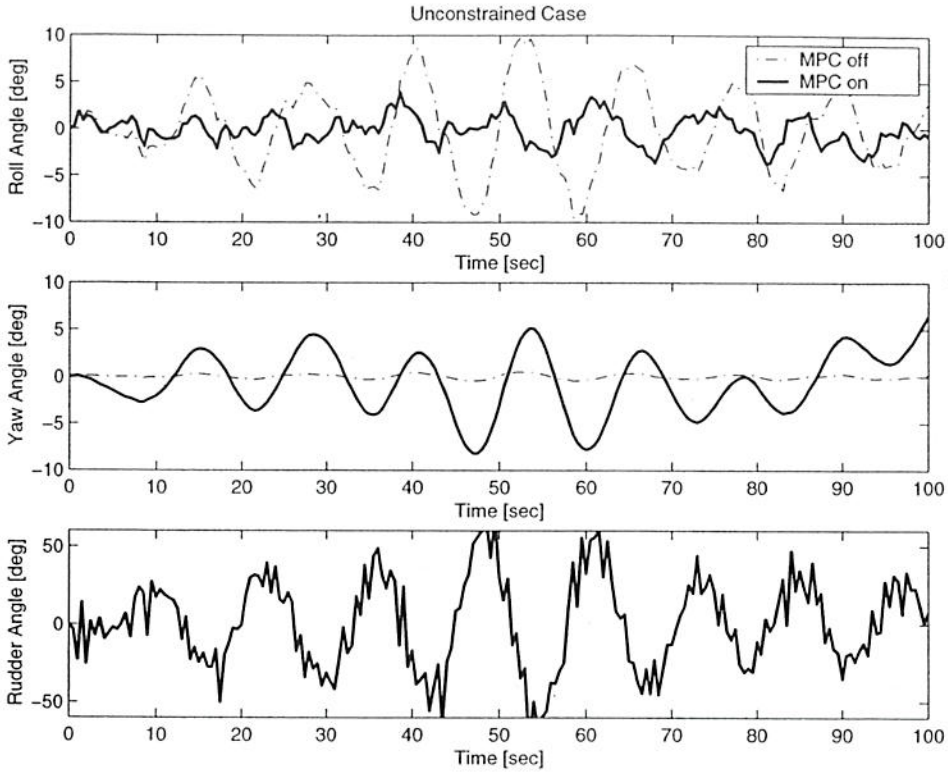

Figure 23.16. Ship motion with no constraints on rudder motion

Figure 23.16 shows the results for the case in which no constraints are applied to the rudder amplitude or slew rate. In this case, a 66% reduction in the standard deviation of the roll angle was achieved relative to no stabilization. The price paid for this is a modest increase in the standard deviation in the yaw angle. Notice, also, that the rudder angle reaches 60 degrees and has a very fast slew rate.

Figure 23.17 shows the results for a maximum rudder angle of 30 degrees and a maximum slew rate of 15 deg/sec. In this case, a 59% reduction in the standard deviation of the roll angle was achieved relative to no stabilization. Notice that the rudder angle satisfies the given constraints via the application of the MPC algorithm.

Figure 23.18 shows the results for a maximum rudder angle of 20 degrees and maximum slew rate of 8 deg/sec. In this case, a 42% reduction in the standard deviation of the roll angle was achieved relative to no stabilization. Notice that the more severe constraints on the rudder are again satisfied via the application of the MPC algorithms.

□□□

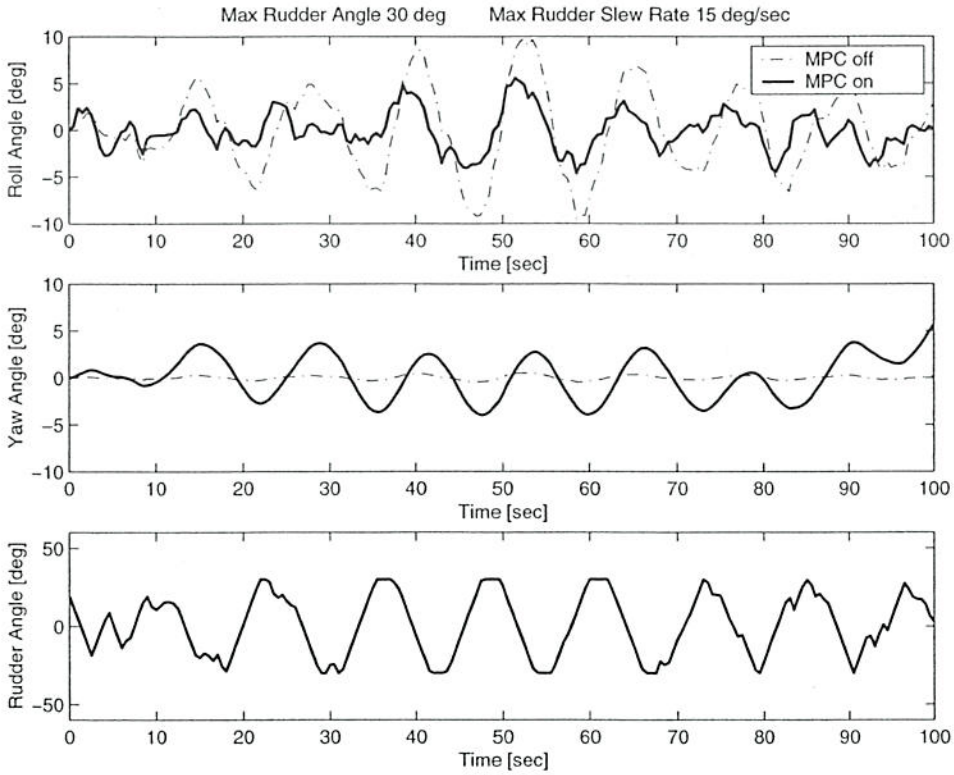

Figure 23.17. Ship motion when rudder is constrained to maximum angle of 30 degrees and maximum slew rate of 15 degrees/sec

23.8 Summary

- MPC provides a systematic procedure for dealing with constraints (both input and state) in MIMO control problems.
- It has been widely used in industry.
- Remarkable properties of the method can be established–e.g., global asymptotic stability provided that certain conditions are satisfied (e.g., appropriate weighting on the final state).
- The key elements of MPC for linear systems are
  - state space (or equivalent) model,
  - on-line state estimation (including disturbances),
  - prediction of future states (including disturbances),
  - on-line optimization of future trajectory subject to constraints by using Quadratic Programming, and
  - implementation of the first step of the control sequence.

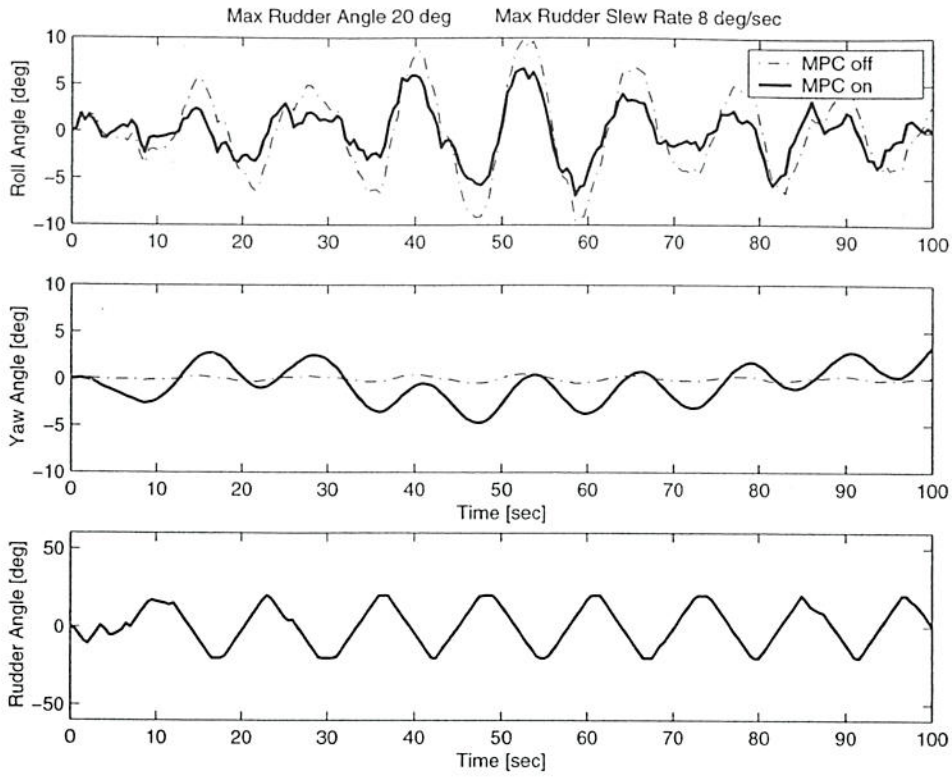

Figure 23.18. Ship motion when rudder is constrained to maximum angle of 20 degrees and maximum slew rate of 8 degrees/sec

23.9 Further Reading

Optimal control background

Bellman, R. (1957). *Dynamic Programming.* Princeton University Press.

Kalman, R.E. (1960). Contributions to the theory of optimal control. *Boletín Sociedad Matemática Mexicana*, 5:102-119.

Lee, E. and Markus, L. (1967). *Foundations of Optimal Control Theory.* Wiley, New York.

Receding-horizon Model Predictive Control

Cutler, C. and Ramaker, B. (1989). Dynamic matrix control. A computer control algorithm. *Proceedings Joint Automatic Control Conference*, San Francisco, California.

Garcia, C.E. and Morshedi, A. (1986). Quadratic programming solution of dynamic matrix control (QDMC). *Chemical Engineering Communications*, 46:73-87.